

Milestone 3 Report

[[Google Drive link to the top-level m3 folder](#)]

Baseline:

Fused:

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.603637 ms	0.335028 ms	0.938665 ms	0.86
1000	5.19709 ms	3.16562 ms	8.36271 ms	0.886
10000	51.2606 ms	31.4453 ms	82.7059 ms	0.8714

Unrolling:

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	1.69302 ms	1.14903 ms	2.84205 ms	0.86
1000	9.03128 ms	7.82228 ms	16.85356 ms	0.886
10000	66.8667 ms	57.9597 ms	124.8264 ms	0.8714

Req_0: Streams

[\[Google Drive link to subfolder req_0\]](#)

- **How does this optimization theoretically optimize your convolution kernel? Expected behavior?**

The point of streams is to stop doing things one at a time. In a single stream version, the H2D copy runs, then the kernel runs, then the D2H copy runs, and only one of those is using its hardware path at any moment. The DMA engines sit idle while the SMs work, the SMs sit idle while the DMA copies. Splitting the batch into chunks and putting each chunk on its own stream lets you pipeline them. Stream 0's matmul can run at the same time as stream 1's H2D copy and stream 2's D2H copy.

For this to actually overlap, the host buffers have to be pinned. Pageable memory forces an internal staging copy that serializes everything. PMPP Chapter 20 covers the same overlap pattern for MPI in section 20.5, the CUDA stream version of the argument is the same.

I expected an improvement but I knew the op time number was going to look weird. The course harness only times what's inside `conv_forward_gpu`, and once the work moves to the prolog, `conv_forward_gpu` is empty so op times are going to be around 0.

- **How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.**

I moved everything into `conv_forward_gpu_prolog` so `conv_forward_gpu` is now empty:

```
__host__ void GPUInterface::conv_forward_gpu(float *device_output, const float
*device_input, const float *device_mask, const int Batch, const int Map_out, const int
Channel, const int Height, const int Width, const int K)
{
    // req_0 all work is done in conv_forward_gpu_prolog
}
```

The prolog pins the host buffers up front:

```
// cudaHostRegister allows for overlap
cudaHostRegister((void*)host_input, inputsz, cudaHostRegisterDefault);
cudaHostRegister((void*)host_output, outputsiz, cudaHostRegisterDefault);
```

Then the per-stream pipeline:

```
// pipeline
for (int s = 0; s < NUM_STREAMS; s++) {
    const int b_start = s * chunk;
    const int input_offset = b_start * Channel * Height * Width;
    const int output_offset = b_start * Map_out * Height_out * Width_out;
    //copy this chunk's slice of input
    cudaMemcpyAsync(*device_input_ptr + input_offset, host_input + input_offset,
chunk_in_bytes, cudaMemcpyHostToDevice, streams[s]);
    // unroll kernel
    int unroll_threads = chunk * Height_out * Width_out;
    int unroll_blocks = (unroll_threads + UNROLL_BLOCK_SIZE - 1) / UNROLL_BLOCK_SIZE;
    matrix_unrolling_kernel<<<unroll_blocks, UNROLL_BLOCK_SIZE, 0,
streams[s]>>>(*device_input_ptr + input_offset, unrolled_chunks[s], chunk, Channel, Height,
Width, K);
    //matmul => mask * unrolled -> matmul_chunks[s]
    dim3 mm_grid((chunk_W_unrolled - 1) / MATMUL_TILE_WIDTH + 1, (Map_out - 1) /
MATMUL_TILE_WIDTH + 1, 1);
    dim3 mm_block(MATMUL_TILE_WIDTH, MATMUL_TILE_WIDTH, 1);
    matrixMultiplyShared<<<mm_grid, mm_block, 0, streams[s]>>>(*device_mask_ptr,
unrolled_chunks[s], matmul_chunks[s], Map_out, Height_unrolled, Height_unrolled,
chunk_W_unrolled, Map_out, chunk_W_unrolled);
    // permute output buffer
    const int out_image_size = Height_out * Width_out;
    dim3 pm_grid((out_image_size - 1) / PERMUTE_BLOCK_SIZE + 1, chunk, 1);
    matrix_permute_kernel<<<pm_grid, PERMUTE_BLOCK_SIZE, 0, streams[s]>>>(matmul_chunks[s],
*device_output_ptr + output_offset, Map_out, chunk, out_image_size);
    // copy back to host
    cudaMemcpyAsync(host_output + output_offset, *device_output_ptr + output_offset,
chunk_out_bytes, cudaMemcpyDeviceToHost, streams[s]);
}
// wait for all streams to finish
for (int s = 0; s < NUM_STREAMS; s++) {
    cudaStreamSynchronize(streams[s]);
    // free per stream
}
```

Each stream gets its own scratch buffers for the unrolled matrix and the matmul output so writes from different streams don't collide. I picked 4 streams because the A40 has separate H2D and D2H copy

engines, and 4 felt like enough to keep both engines plus the SMs busy without making the per-chunk matmul so small that launch overhead would dominate. Accuracy was the same as baseline.

- **Did the performance match your expectation? Explain why or why not, by analyzing profiling results.**

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.001553 ms	0.001764 ms	0.003317 ms	0.86
1000	0.003858 ms	0.002895 ms	0.006753 ms	0.886
10000	0.002805 ms	0.003236 ms	0.006041 ms	0.8714

The op time table is misleading on purpose. Op times of 0.003 ms aren't the kernels running in microseconds, that's just the time inside `conv_forward_gpu`, which is empty. The real GPU work is in the prolog. The Layer Time line in the same run gives you the actual end to end number, 463.5 ms for Conv1 and 398.3 ms for Conv2.

```
[6/8] Executing 'cuda_gpu_kern_sum' stats report
```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	
51.2	57569492	8	7196186.5	7180593.0	7144291	7297761	52094.1	matrixMultiplyShared(const f
42.7	47969457	8	5996182.1	5945796.0	5280322	6874177	649554.6	matrix_unrolling_kernel(const
6.1	6818978	8	852372.3	854320.0	733920	954304	94362.3	matrix_permute_kernel(const
0.0	2400	2	1200.0	1200.0	1184	1216	22.6	do_not_remove_this_kernel()
0.0	2176	2	1088.0	1088.0	1088	1088	0.0	prefn_marker_kernel()

The nsys profile is what to actually look at. Summed across both layers from `cuda_gpu_kern_sum`:

- `matrixMultiplyShared`, 8 instances, 57.6 ms total
- `matrix_unrolling_kernel`, 8 instances, 47.97 ms total
- `matrix_permute_kernel`, 8 instances, 6.82 ms total

```
[7/8] Executing 'cuda_gpu_mem_time_sum' stats report
```

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
72.1	70975986	8	8871998.3	8441266.0	7037283	13285666	2242172.6	[CUDA memcpy Device-to-Host]
27.9	27405545	12	2283795.4	2772833.5	832	4482144	1801224.7	[CUDA memcpy Host-to-Device]

And from `cuda_gpu_mem_time_sum`:

- D2H, 8 transfers, 70.98 ms total
- H2D, 12 transfers, 27.4 ms total

If those ran serially with no overlap, the wall clock would be roughly the sum of all of it, somewhere around 210 ms per layer pair. With 4 streams overlapping, the wall clock approaches whatever stage is the bottleneck instead. The Layer Time numbers say the overlap isn't perfect, there's still some serial time at the head and tail of the pipeline (chunk 0 has nothing to overlap H2D with, chunk 3 has nothing to overlap D2H with), but most of the middle is hidden.

The thing the nsys data makes obvious is that D2H is the single most expensive stage at 70.98 ms, more than the matmul at 57.6 ms. That makes sense given the output size, but it also means streams help even more here than they would in a compute heavy workload. Three quarters of that D2H cost can be hidden behind the next chunk's matmul.

So yes, the optimization does what it's supposed to, the op time metric just can't see it.

- **Does this optimization synergize with any other optimizations? How?**

Streams don't really care what kernel is inside the pipeline. They'd wrap around `op_6` the same way they wrap around the M2 unrolling kernel here.

What matters more is what happens when the matmul gets faster. Right now D2H is almost as expensive as the matmul. Speed up the matmul and D2H becomes the bottleneck, at which point streams matter more, since they're hiding the thing that's now on the critical path.

Streams aren't in my final submission because the rules forbid it for the performance test, not because they conflict with anything.

- **List your references used while implementing this technique. (you must mention textbook pages at the minimum)**

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 20 (Programming a heterogeneous computing cluster), "Overlapping computation and communication," pp. 462-469.

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 6 (Performance considerations), "Hiding memory latency," pp. 133-137.

Harris, M. "How to Overlap Data Transfers in CUDA C/C++." NVIDIA Developer Blog.
<https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/>

Req_1: Tensor Cores

[\[Google Drive link to subfolder req_1\]](#)

- **How does this optimization theoretically optimize your convolution kernel? Expected behavior?**

Tensor cores are dedicated matmul hardware on the SM. On Ampere a single warp can issue an `mma.sync` that does a full $16 \times 16 \times 16$ FP16 matmul into an FP32 accumulator in one instruction. That's $16 \times 16 \times 16 \times 2 = 8192$ FMAs per warp per instruction, compared to the SIMT path where 32 threads do one FMA each per cycle. The throughput ceiling is way higher.

The catch is the data has to be in FP16, has to be arranged into the WMMA fragment layout, and the tile sizes are fixed at 16. So M, N, and K of the matmul all need to be multiples of 16, otherwise you pad up and waste the extra rows or columns.

For my kernel the bad news is immediate. Conv1 has `Map_out=4`, but the WMMA M dimension is forced to 16. That's 12 rows of zero padding for every tile, three quarters of the multiplies are by zero. Conv2 has `Map_out=16` which lines up cleanly, but the K dimension is $\text{Channel} * K * K = 4 * 7 * 7 = 196$, which doesn't divide cleanly into $K=16$ either, so there's some padded waste inside the dot product too.

I expected req_1 to lose on Conv1 and maybe break even on Conv2. Tensor cores wouldn't be that useful for matmuls this small.

- **How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.**

The kernel runs one warp per block, 32 threads in 1D. Each warp computes one 16×16 output tile. Shared memory holds the FP16 tiles, the FP32 to FP16 conversion happens during the load, the inner loop accumulates into an FP32 fragment, and the result writes back through a shared memory landing pad so the (b, m, h, w) permute can happen on the way out:

```
// shared memory tiles
__shared__ __half tileA[WMMA][WMMA];
__shared__ __half tileB[WMMA][WMMA];
```

```

// shared memory landing pad
__shared__ float tileOut[WMMA][WMMA];
// initialize fragments
wmma::fragment<wmma::matrix_a, WMMA, WMMA, WMMA, __half, wmma::row_major> a_frag;
wmma::fragment<wmma::matrix_b, WMMA, WMMA, WMMA, __half, wmma::row_major> b_frag;
wmma::fragment<wmma::accumulator, WMMA, WMMA, WMMA, float> c_frag;
wmma::fill_fragment(c_frag, 0.0f);

int num_tiles = (H_unrolled + WMMA - 1) / WMMA;
// 32 lanes 256 elements -> 256/32 = 8 elements per lane
for (int t = 0; t < num_tiles; t++) {

    for (int e = tx; e < WMMA*WMMA; e += 32){
        int local_row = e / WMMA, local_col = e % WMMA;
        // bounds check, fp32 to fp16
        tileA[local_row][local_col] = __float2half(mask[a_row * H_unrolled +
a_col]);
    }
    // same pattern for tileB with implicit unroll indexing
    // ...
    __syncthreads();
    wmma::load_matrix_sync(a_frag, &tileA[0][0], WMMA);
    wmma::load_matrix_sync(b_frag, &tileB[0][0], WMMA);
    wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
    __syncthreads();
}
wmma::store_matrix_sync(&tileOut[0][0], c_frag, WMMA, wmma::mem_row_major);

```

Out of bounds rows and columns get zero filled on load, that's what makes the kernel work for any M and K. It's also exactly where the wasted work on Conv1 comes from.

The FP32 accumulator inside c_frag keeps the math precise even though the multiplies are FP16. Accuracy matches baseline.

- **Did the performance match your expectation? Explain why or why not, by analyzing profiling results.**

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	2.30406 ms	0.382667 ms	2.686727 ms	0.86

1000	5.3791 ms	3.52274 ms	8.90184 ms	0.886
10000	51.8768 ms	34.6114 ms	86.4882 ms	0.8714

Slower on both convolutions, worse on Conv2 than Conv1. The Conv2 regression caught me off guard, I figured Conv2 at least would break even.

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	33.33	Block Limit Registers [block]	48
Theoretical Active Warps per SM [warp]	16	Block Limit Shared Mem [block]	21
Achieved Occupancy [%]	33.10	Block Limit Warps [block]	48
Achieved Active Warps per SM [warp]	15.89	Block Limit SM [block]	16

The 4.00 theoretical warps per scheduler this kernel can issue according to its occupancy are below the hardware maximum of 12. This kernel's theoretical occupancy (33.3%) is limited by the number of blocks that can fit on the SM.

Theoretical Occupancy: 33.33%
Est. Local Speedup: 66.67%

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	4,000,000	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	40	Static Shared Memory Per Block [kbyte/block]	2.05
Block Size	32	Dynamic Shared Memory Per Block [kbyte/block]	0
Threads [thread]	128,000,000	Driver Shared Memory Per Block [kbyte/block]	1.02
Waves Per SM	2,976.19	Shared Memory Configuration Size [kbyte]	65.54
Uses Green Context	0	Stack Size	1,024
# SMs [SM]	84	# IPCs	42
Enabled TPC IDs	all		-

The NCU data flips most of my initial guess. Achieved occupancy is 33.10%, basically pinned to the theoretical max of 33.33%, and the limiter is registers. Each thread uses 40 registers and there are 32 threads per block, so the kernel can only fit a handful of blocks per SM at once. NCU flags this with an estimated 66.67% local speedup left on the table if occupancy got fixed.



The Compute Workload panel is what really tells me where the time is going. The Tensor (FP) pipe is at about 3% utilization. ALU is at 60%, FMA at 30%, and NCU calls ALU the bottleneck. So the tensor cores are barely doing anything, the kernel is spending almost all its time on integer index math

and the bookkeeping for the implicit unroll loads, and the `mma.sync` gets to fire only occasionally. I built a tensor core kernel that doesn't really use the tensor cores.

Conv1 has the extra problem that `Map_out=4` forces the M dimension up to 16 with 12 padded zero rows, so 75% of every mma is multiplying by zero. Combined with the ALU bottleneck, Conv1 ends up just barely worse than baseline.

Conv2 fits the M dimension cleanly but `K=196` rounds up to 208 (13 tiles of 16), so a small 6% waste there. The bigger issue is the same one as Conv1, the indexing pipe is the bottleneck, not the tensor pipe. With ALU saturated and tensor at 3%, all the WMMA setup is basically wasted overhead.

If I were going to push on this, I'd cut the per-block ALU work first. Combine multiple warps per block so the load loop amortizes, and try to simplify the implicit unroll indexing so less integer math runs in the inner loop. Just adding more warps for occupancy wouldn't fix it on its own because the bottleneck isn't latency, it's the ALU pipe being saturated by indexing instead of by useful compute. For shapes this small (4x16 effective on Conv1, 16x16 on Conv2) the WMMA unit is too big a primitive to be worth the setup cost.

- **Does this optimization synergize with any other optimizations? How?**

It doesn't really stack with much. `const restrict` from `op_1` still applies. Loop unrolling from `op_2` doesn't do anything because there's no inner FMA loop anymore, the `mma.sync` replaces the whole thing. Putting the mask in constant memory like `op_0` still works but doesn't help with the actual problems, which are the low occupancy and the wasted M rows on Conv1.

The thing that would actually help is putting more warps in each block so the SMs aren't half empty. That's a rewrite of the kernel though, not something I can stack on.

Tensor cores are built for big matmuls with sizes that are clean multiples of 16. My Conv1 is `M=4` and my Conv2 has `K=196`, neither of those fits, so I'm using hardware that's designed for transformer attention shapes on a kernel that doesn't look like that at all.

- **List your references used while implementing this technique. (you must mention textbook pages at the minimum)**

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 16 (Deep learning), "Formulating a convolutional layer as GEMM," pp. 379-384.

NVIDIA Corporation. "Programming Tensor Cores in CUDA 9." NVIDIA Developer Blog.
<https://developer.nvidia.com/blog/programming-tensor-cores-cuda-9/>

NVIDIA Corporation. "CUDA C++ Programming Guide," Section 10.24 Warp Matrix Functions.
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

Lecture 17: Tensor Operations, ECE 408 SP26.

Op_0: Weight matrix (Kernel) in constant memory

[\[Google Drive link to subfolder op_0\]](#)

- **How does this optimization theoretically optimize your convolution kernel? Expected behavior?**

Constant memory is a 64 KB region that lives off-SM but has its own dedicated cache on each SM. The cache is built specifically for broadcast access, when every thread in a warp reads the same address, the cache serves all 32 lanes from a single fetch. For the conv mask that's a perfect fit, the mask is small, read-only for the lifetime of the kernel, and within a tile a lot of threads end up wanting the same mask weight at the same time.

My masks are small enough to fit easily. Conv1 is $4 * 1 * 7 * 7 * 4$ bytes = 784 B, Conv2 is $16 * 4 * 7 * 7 * 4$ bytes = 12,544 B. Both well under 64 KB.

What I expected: a small win on both layers because the mask loads are now coming from a cache that's sitting closer to the SMs and that handles broadcast for free. PMPP Chapter 7 walks through this exact pattern for convolution masks.

- **How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.**

Declared a constant array sized for the worst case (Conv2), copied the mask in during the prolog, and removed the mask pointer parameter from the kernel since it now reads from the global symbol:

```
#define MAX_MASK_SIZE (16 * 4 * 7 * 7)
__constant__ float const_mask[MAX_MASK_SIZE];

// removed mask param because the kernel now reads from const_mask
__global__ void matmul_conv_fused(const float *input, float *output,
                                int Batch, int Map_out, int Channel, int Height, int Width,
                                int K)
{
    // ...
    if (Row < Map_out && a_col < H_unrolled) {
        tileA[ty][tx] = const_mask[Row * H_unrolled + a_col]; // was global mem
    }
}
```

```
// ...
}
```

The prolog copies the mask in with `cudaMemcpyToSymbol` instead of a regular `cudaMemcpy`:

```
cudaMemcpyToSymbol(const_mask, host_mask, masksz);
```

Everything else is the same as the M2 fused baseline. Accuracy is identical to baseline.

- **Did the performance match your expectation? Explain why or why not, by analyzing profiling results.**

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.543768 ms	0.500486 ms	1.044254 ms	0.86
1000	5.08864 ms	4.83516 ms	9.9238 ms	0.886
10000	50.3468 ms	48.0188 ms	98.3656 ms	0.8714

Conv1: 50.35 ms vs 51.26 baseline, about 2% faster. Conv2: 48.02 ms vs 31.45 baseline, about 53% slower.

This was not what I expected at all. Same kernel, same change, opposite outcomes on the two layers. Conv1 got the small win I was looking for. Conv2 fell off a cliff.

Constant memory helped on Conv1 (`Map_out=4`, all threads in a block read from at most 4 mask rows, strong broadcast behavior) but hurt on Conv2 (`Map_out=16`, threads in one block span 16 mask rows, broadcast serialization). Same kernel, opposite results, the access pattern is the deciding factor, not the mask size or read-only-ness.

The reason is the broadcast assumption breaks on Conv2. Constant cache is fast when all threads in a warp want the same address, and it serializes when they want different addresses. Conv1 has `Map_out=4`, so within a tile the 16 rows of threads cluster onto only 4 distinct mask rows, lots of broadcast hits. Conv2 has `Map_out=16`, so all 16 rows of threads in a tile want different mask rows, and

the cache has to serialize one address per cycle instead of broadcasting. With 16 different addresses per warp, that's roughly 16x the latency of the L1/global path that the baseline was using.

So the big win I expected on Conv2 turned into a big loss because the access pattern doesn't match what constant memory is built for. The L1 cache that the global path was using doesn't care if addresses differ across the warp, it serves them through normal coalescing. Constant memory is faster only when the broadcast actually applies.

This was a useful negative result. Constant memory isn't free, it's a tradeoff that depends on the access pattern, and you can't tell which side of the tradeoff you're on without running both versions.

- **Does this optimization synergize with any other optimizations? How?**

Doesn't really stack with anything I'm using because of the Conv2 regression. The total at 98.37 ms is worse than the 82.71 ms M2 baseline, so even if I composed it with op_1 or op_2 the total would still be slower than just keeping the mask in global memory.

If the network had a different shape where every layer had small Map_out (like 4 everywhere), constant memory would compose cleanly with the other optimizations and give a real win across the board. For this specific LeNet variant the access pattern on Conv2 makes it not worth carrying.

- **List your references used while implementing this technique. (you must mention textbook pages at the minimum)**

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 7 (Convolution: An introduction to constant-memory and caching), "Constant memory and caching," pp. 159-162.

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 5 (Memory architecture and data locality), "CUDA memory types," pp. 96-102.

NVIDIA Corporation. "CUDA C++ Programming Guide," Section 5.3 Memory Hierarchy and 6.2 Cuda Runtime. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

Op_1: __restrict__ keyword

[[Google Drive link to subfolder op_1](#)]

- **How does this optimization theoretically optimize your convolution kernel? Expected behavior?**

`__restrict__` tells the compiler that the pointers I'm passing in don't alias each other. If the compiler knows that, then every store through output could possibly land somewhere the mask or input is reading from, so it can't keep any loaded value in a register across a store - it has to reload from global memory. Once I add `__restrict__`, that issue goes away and the compiler is free to hoist loads out of the loop and keep things in registers.

The other half is the `const` qualifier on the read-only pointers. With `const + __restrict__` the compiler is allowed to route those loads through a non-coherent cache path. In SASS that usually shows up as LDG.E.CI (read-only data cache) or, when the compiler can prove the address is uniform across a warp, LDG.E.CONSTANT (constant cache, broadcast to all lanes). Either way it's separate from L1 and avoids the coherence assumptions a regular LDG.E would have to make.

I wasn't expecting 2x though. My fused kernel pulls each input/mask tile into shared memory once and then does 16 multiply-accumulates per thread out of shmem, so the global load cost is not the dominant thing. I figured maybe a percent or two, and that it definitely shouldn't make anything worse since it's a free hint.

- **How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.**

One change. I added `__restrict__` to the three pointer parameters of the fused kernel:

```
// added op1: __restrict__ added to all 3 pointer params. Tells compiler that mask, input and
output don't alias each other in memory

// so they won't overlap, also routes global loads through read only data cache up to 2x
improvement
```

```
__global__ void matmul_conv_fused(const float * __restrict__ mask, const float * __restrict__
input, float * __restrict__ output, int Batch, int Map_out, int Channel, int Height, int
Width, int K)
```

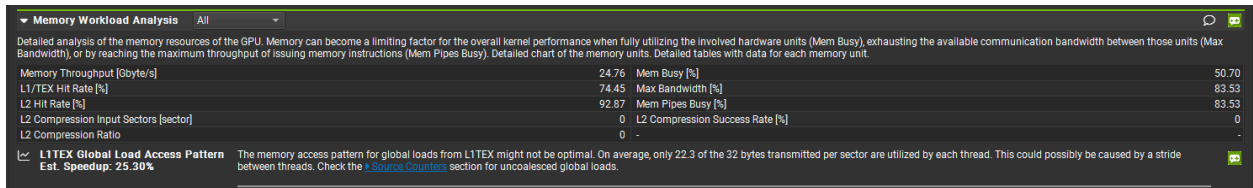
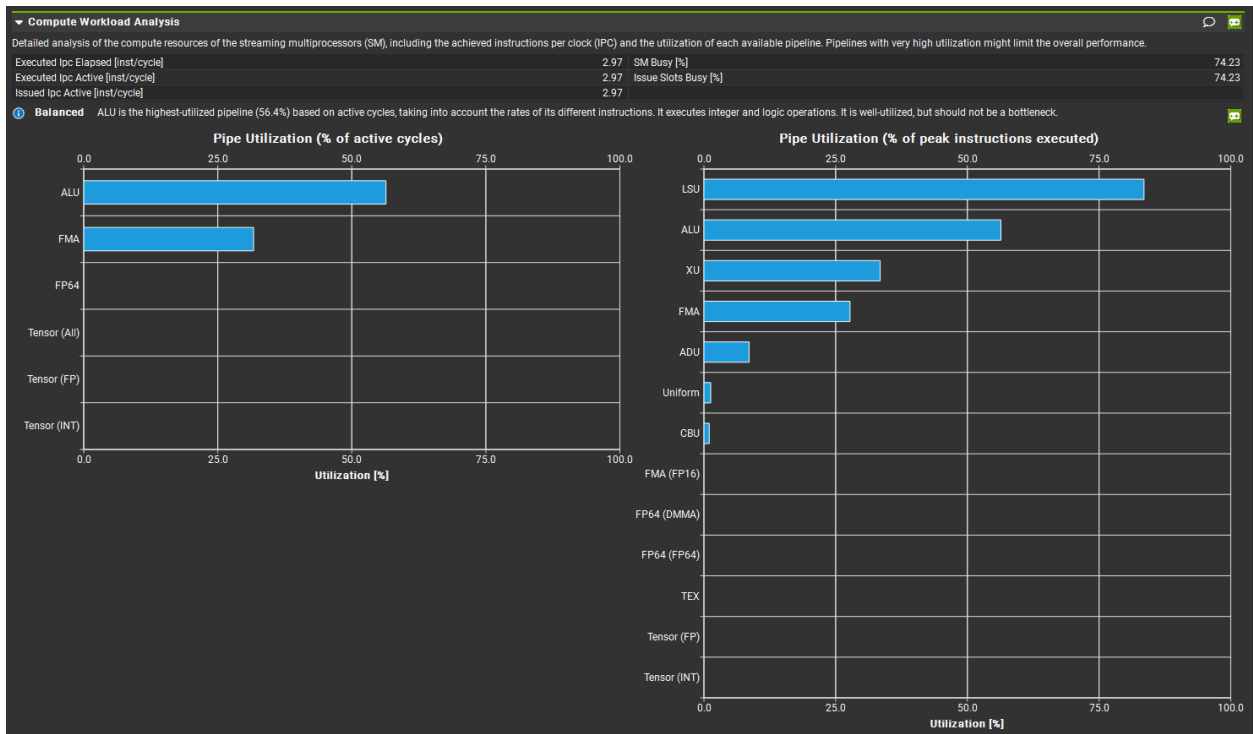
Everything else such as host code, kernel body, the tiled matmul loop, is the same as my M2 fused version.

Correctness is fine. Accuracy is the same as baseline as shown below. That's expected because `__restrict__` doesn't change what the code does, just what the compiler is allowed to assume.

- **Did the performance match your expectation? Explain why or why not, by analyzing profiling results.**

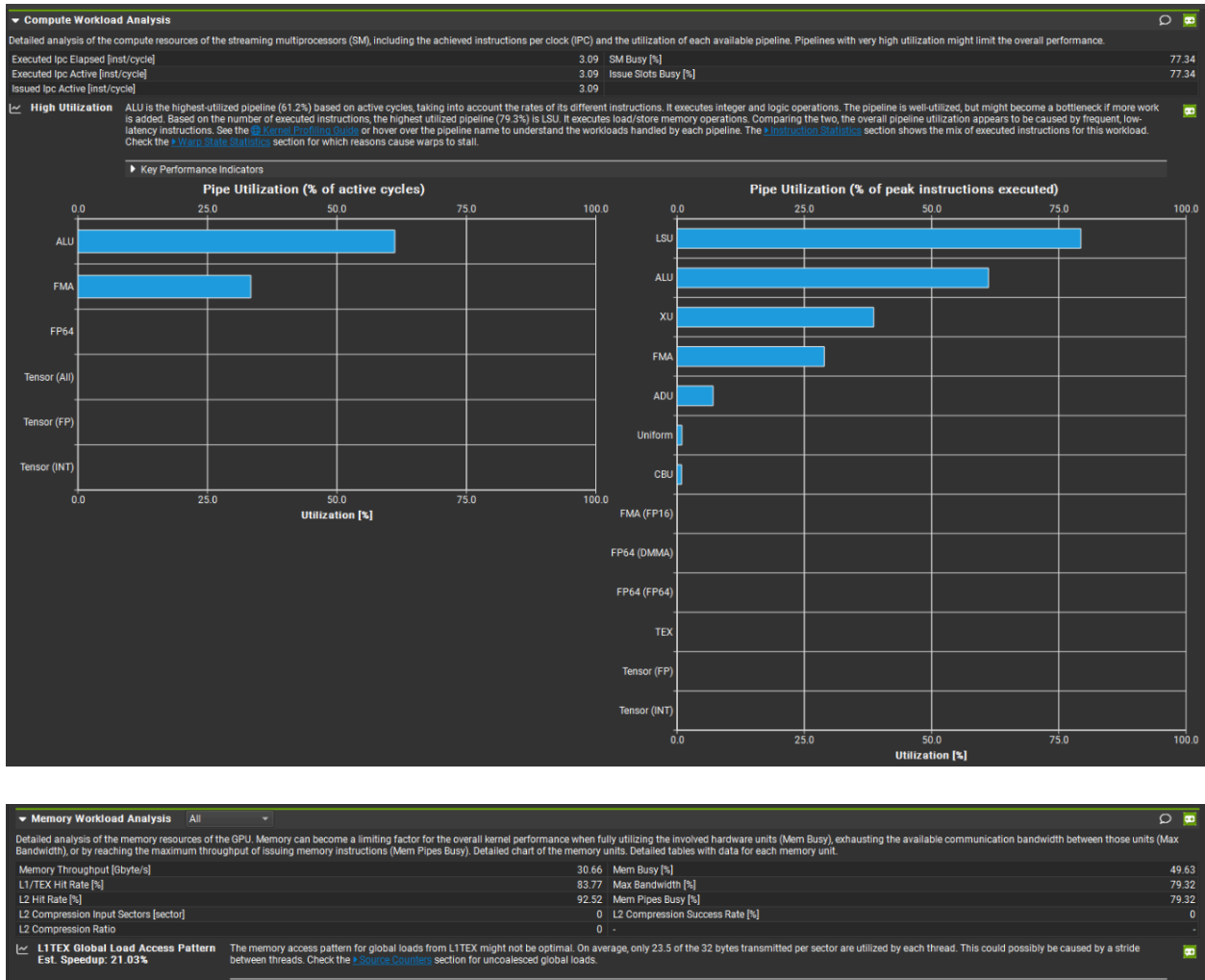
Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	1.2387 ms	0.335308 ms	1.574008 ms	0.86
1000	5.2348 ms	3.17777 ms	8.41257 ms	0.886
10000	51.1672 ms	31.4618 ms	82.629 ms	0.8714

Convolution 1:



174	00007f33 c527bcd0	IMAD R4, R4, c[0x0][0x100], R7	FMA, FRAHEAVY	21	20	0.33%	Integer	0.45%	31
175	00007f33 c527bce0	IMAD.WIDE R4, R4, RS, c[0x0][0x100]	FMA, FRAHEAVY	20	21	0.31%	Integer	0.45%	31
176	00007f33 c527bcf0	LDG.E.CONSTANT R8, [R4-64]	LSU	21	19	3.96%	Load/Store	0.45%	31
177	.L_x.2 00007f33 c527bd00	BSYNC B0	CBU	19		0.15%	Control	0.60%	31
178	.L_x.1 00007f33 c527bd10	STS [R27+0x400], R8	LSU	19		7.78%	Load/Store	0.58%	32
179	00007f33 c527bd20	UIAD03 UR4, UR4, 0x1, UR2	UNIFORM	18		0.04%	Uniform Data...	0.58%	32

Convolution 2:



Conv1 numbers. L1/TEX hit rate is 74.45%, L2 is 92.87%, memory throughput 24.76 GB/s. Mem pipes are busy at 83.53% but DRAM barely shows up, so basically everything the kernel asks for is already cached. That's the shared-memory reuse pattern, each tile gets pulled in once and reused 16 times per thread. NCU flags an estimated 25.30% speedup from fixing uncoalesced loads (22.3 of 32 bytes per sector used) but that's a layout problem, not something `__restrict` can fix.

NCU labels Conv1 "High Utilization." ALU is at 61.2% of active cycles, FMA around 28%, LSU peak 79.3%, SM Busy 77.34%. The bottleneck is LSU and integer index math, not the multiply-accumulates, which makes sense given the M=16 tile only has 4 productive rows on Conv1 (Map_out=4, 12 rows of zero-padding waste).

Conv2 looks similar but cleaner. L1/TEX hit rate jumps to 83.77%, L2 is 92.52%, throughput 30.66 GB/s. The higher hit rate is because the tile is fully populated (Map_out=16), so every load gets reused by 16 productive threads instead of 4. NCU calls it "Balanced" with ALU 56.4%, FMA ~28%, LSU peak ~79%, SM Busy 74.23%. SM Busy is actually a bit lower than Conv1 because Conv2 has less total work to do, not because the SMs are more efficient per cycle. Same uncoalesced-load hint shows up here too at 21.03%.

Both layers tell the same story. High L1/TEX hit rate, low DRAM use, FMA nowhere near saturated. That's what you'd want a fused tiled matmul to look like, and it's also why an aliasing hint barely changes anything, because the loads it would reroute are already cheap. The SASS confirms this. The mask loads show up as LDG.E.CONSTANT (line 176, LDG.E.CONSTANT R8, [R4.64], ~3.96% of stalls), so the compiler decided the addresses are warp-uniform and used constant-cache broadcast instead of the read-only data cache (LDG.E.CI) that the Appleyard post talks about. Constant-cache broadcast is actually the better outcome for this access pattern, which is why the per-warp cost was already low and why the speedup ended up as ~1% on Conv1 and doesn't affect Conv2.

Compared to my M2 baseline at batch 10000 (Op1: 51.65 ms, Op2: 31.45 ms):

- Conv1 went from 51.65 → 51.17 ms, about 1% faster
- Conv2 went from 31.45 → 31.46 ms, basically no change

Direction is right, magnitude is small, no regression. Which is what I expected. The Appleyard post got a 2× because his example kernel is essentially streaming reads - every load goes through the read-only path. Mine isn't. Each tile gets pulled into shared memory once and then reused 16 times per thread, so global loads are a small slice of the total cost. Cutting their cost by some percent doesn't move the kernel-level number much. In my SASS the heavy loads showed up as LDG.E.CONSTANT rather than LDG.E.CI, meaning the compiler decided the addresses are warp-uniform and used constant-cache broadcasting.

- **Does this optimization synergize with any other optimizations? How?**

op_0 (constant memory for mask): Both go after the cost of mask loads. Once the mask is in constant memory, the read-only cache benefit shifts entirely to input. Stacking them should be strictly better than either alone.

op_2 (loop unrolling): Unrolling helps the compiler schedule more independent loads, but it can only do that if it knows the loads aren't going to be invalidated by intervening stores - which is exactly what `__restrict__` guarantees.

req_1 (Tensor Cores): WMMA fragment loads benefit from the same non-aliasing guarantees, so `__restrict__` should still help even after switching to tensor cores.

There's no real downside to having it - it's a free compiler hint with no runtime cost, so I'm planning to keep it on for the stacked optimization.

- **List your references used while implementing this technique. (you must mention textbook pages at the minimum)**

Appleyard, J. "CUDA Pro Tip: Optimize for Pointer Aliasing." NVIDIA Developer Blog.
<https://developer.nvidia.com/blog/cuda-pro-tip-optimize-pointer-aliasing/>

NVIDIA Corporation. "CUDA C++ Programming Guide," Section 7.2.6 `__restrict__`.
<https://docs.nvidia.com/cuda/archive/12.8.1/cuda-c-programming-guide/index.html>

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 6 (Performance considerations), "A checklist of optimizations," pp. 141-143.

Op_2: Loop unrolling

[[Google Drive link to subfolder op_2](#)]

- **How does this optimization theoretically optimize your convolution kernel? Expected behavior?**

The idea is to take the inner accumulation loop (the one that does the 16 multiply-accumulates per tile) and tell the compiler to fully unroll it. Without unrolling, every iteration has a loop counter increment, a comparison, and a conditional branch. Unrolling gets rid of all of that and just lays out 16 sequential FFMA instructions back to back. No branches, no loop overhead, and the compiler has more freedom to schedule those FFMA's and the loads that feed them, since it can see the whole thing flat instead of one iteration at a time.

I wasn't expecting much in absolute terms. TILE_WIDTH is 16, which is small, so the loop overhead per iteration is already tiny relative to the FFMA itself. And TILE_WIDTH is a compile-time constant, so nvcc was probably already unrolling this loop on its own. The pragma is more of a guarantee than a request. I figured maybe a percent or two if anything, and zero risk because if the compiler was already doing it then nothing will change.

- **How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.**

One change. Added #pragma unroll in front of the inner accumulation loop in the fused kernel:

```
// op 2: Loop unrolling explicit full unroll of inner accumulation loop.

// TILE_WIDTH=16 is constant so this expands into

// 16 independent FFMA's with no loop counter or branch overhead.

// basically just tells the compiler to unroll into 16 FMAS

#pragma unroll

    for (int i = 0; i < TILE_WIDTH; i++) {

        acc += tileA[ty][i] * tileB[i][tx];
```

```
}

```

Everything else is the exact same as before. Accuracy is the same as baseline. Unrolling can't change what the loop computes, only how the instructions get laid out.

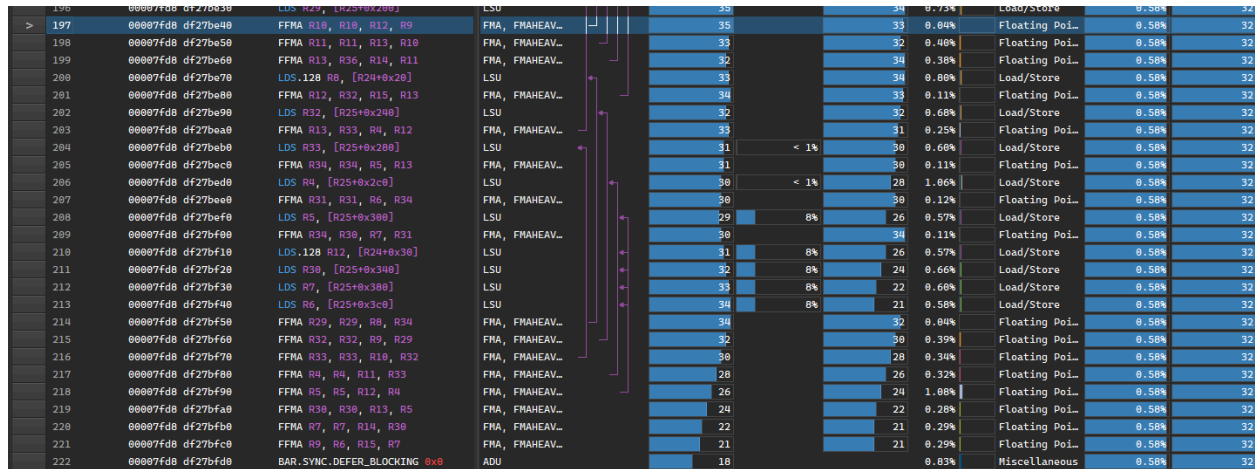
- **Did the performance match your expectation? Explain why or why not, by analyzing profiling results.**

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	1.08246 ms	0.334516 ms	1.416976 ms	0.86
1000	5.22951 ms	3.16399 ms	8.3935 ms	0.886
10000	51.3822 ms	31.445 ms	82.8272 ms	0.8714

Compared to op_1 at batch 10000 (Op1: 51.17 ms, Op2: 31.46 ms):

- Conv1 went 51.17 to 51.38 ms basically no change
- Conv2 went 31.46 to 31.45 ms, also a wash

So no measurable speedup, which lines up with what I expected. The most likely explanation is that nvcc was already unrolling this loop without the pragma, since TILE_WIDTH is a compile-time constant equal to 16 and the trip count is small. The pragma is just making the unroll explicit. The other piece is that the inner loop wasn't the bottleneck in the first place. The op_1 NCU data showed FMA pipe at around 28% utilization and SM busy in the mid-70s, with LSU and ALU as the dominant pipes. Removing branch overhead from a loop that wasn't holding back FMA throughput doesn't move the kernel-level number.



The SASS confirms the unroll happened. The inner accumulation region shows a long run of FFMA instructions back to back (lines 197 through 221 in the screenshot) with LDS loads interleaved to feed them, and no BRA branch anywhere in the body. The loop is completely flat. You can also see the compiler hoisting LDS loads from later iterations up into the FFMA stream around lines 200 to 213, which is the kind of instruction-level scheduling unrolling enables. The stall percentages on those FFMAs are all under about 1%, which says the FFMA pipe isn't backed up. So the unroll did what it was supposed to do, the kernel just doesn't benefit because FMA wasn't the bottleneck (op_1 NCU had FMA at around 28%).

- **Does this optimization synergize with any other optimizations? How?**

It pairs with op_1 (`__restrict`) in the sense that unrolling exposes a longer sequence of independent FFMAs and loads, which the compiler can only reorder freely if it knows none of those loads can be invalidated by stores in between. `__restrict` is what gives it that guarantee. Without `__restrict` the unrolled body would still execute, but the compiler would be more conservative about scheduling.

It also pairs with op_6 (joint register and shared memory tiling), where each thread accumulates into a register array of size `U_OUT`. Those per-thread inner loops benefit from the same unroll treatment, and that's where unrolling actually starts paying off because there are more FFMAs per thread to schedule.

No real downside to using it anywhere.

- **List your references used while implementing this technique. (you must mention textbook pages at the minimum)**

NVIDIA Corporation. "CUDA C++ Programming Guide," Section 10.40 #pragma unroll.

<https://docs.nvidia.com/cuda/archive/12.8.1/cuda-c-programming-guide/index.html>

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 6 (Performance considerations), "A checklist of optimizations," pp. 141-143.

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 6 (Performance considerations), "Hiding memory latency," pp. 133-137.

Op_3: Parameter sweep (Tile width and Coarsening factor)

[\[Google Drive link to subfolder op_3\]](#)

- **How does this optimization theoretically optimize your convolution kernel? Expected behavior?**

I tested 2 things here, tile width and thread coarsening. They go after different things.

Tile width is about the shared-memory tile size and the block shape. Bigger tiles mean each tile load is amortized over more FMAs per thread, so the global-memory-to-FMA ratio improves. But bigger tiles also mean more shared memory per block, fewer blocks resident per SM, lower occupancy, and worse latency hiding. Smaller tiles do the opposite. So there's a sweet spot, and where it lands depends on the kernel's exact mix of memory and compute. The right way to find it is to sweep.

Thread coarsening is when each thread does more work, specifically computes multiple output columns instead of just one. The block grid in X gets divided by the coarsening factor, and each thread keeps an accumulator array of size COARSE in registers. The win is that the A tile (the mask tile) only has to be loaded once and gets reused across COARSE different B tiles, so you save on redundant tile loads. The cost is more registers per thread, which can drop occupancy if it pushes you past the limit. Same story as tile width, sweep it.

I expected tile width 16 to be the winner since that's been the baseline all along. For coarsening I expected 2 or 4 to help and 8 to start hurting from register pressure.

- **How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.**

For the tile width sweep I just changed the #define at the top. Nothing else changed.

```
#define TILE_WIDTH 16 // swept 4, 8, 16, 32
```

For coarsening, each thread now accumulates COARSE separate output columns. The B tile gets widened to hold COARSE column-tiles side by side, the load loop runs COARSE times, the inner compute does a nested COARSE loop, and the writeback also loops COARSE times.

```
#define TILE_WIDTH 16

#define COARSE 4

__shared__ float tileA[TILE_WIDTH][TILE_WIDTH];

__shared__ float tileB[TILE_WIDTH][COARSE*TILE_WIDTH];

float acc[COARSE];

for (int c = 0; c < COARSE; c++) acc[c] = 0.0f;

// The inner compute, the part that actually does the work:
// for each K element, multiply against COARSE different B cols
    for (int i = 0; i < TILE_WIDTH; i++) {
        float a_val = tileA[ty][i];
        for (int cc = 0; cc < COARSE; cc++) {
            acc[cc] += a_val * tileB[i][cc * TILE_WIDTH + tx];
        }
    }

// The grid in X also has to shrink by COARSE since each block now covers COARSE column-tiles
instead of one:

    dim3 gridDim((Width_unrolled + COARSE * TILE_WIDTH - 1) / (COARSE * TILE_WIDTH),
(Map_out + TILE_WIDTH - 1) / TILE_WIDTH, 1);
```

Accuracy remained the same for both as shown below.

- **Did the performance match your expectation? Explain why or why not, by analyzing profiling results.**

Tile Width Sweep:

Tile Width	Op Time 1	Op Time 2	Total Execution Time
4	89.3701 ms	228.427 ms	317.7971 ms
8	44.9717 ms	53.7213 ms	98.693 ms
16 (base)	52.1818 ms	31.5358 ms	83.7176 ms
32	131.366 ms	80.8384 ms	212.2044 ms

Tile width 16 won overall, matching expectations. Tile 4 is bad because the per-thread reuse is tiny, you're doing barely any FMAs per shared-memory load. Tile 32 hurts because each block needs $32 \times 32 \times 4$ bytes = 4 KB just for one tile and you've got two tiles, so 8 KB per block, plus $32 \times 32 = 1024$ threads which is the per-block limit. Occupancy gets squeezed and the kernel slows down a lot.

One interesting thing in this sweep, tile 8 actually beats tile 16 on Conv1 (44.97 vs 52.18 ms) but loses badly on Conv2 (53.72 vs 31.54 ms). Conv1 has Map_out=4, so 12 of 16 rows in the M=16 tile are zero-padded waste. With tile 8, only 4 of 8 rows are wasted (still bad but less bad), so less wasted FMA work per block. Conv2 fills the tile cleanly at 16 so the bigger tile pays off. Tile 16 wins on Conv2 by enough to make up for losing to tile 8 on Conv1, so it's the right pick when you have to commit to one.

Tile Width = 16, Coarsening factor sweep:

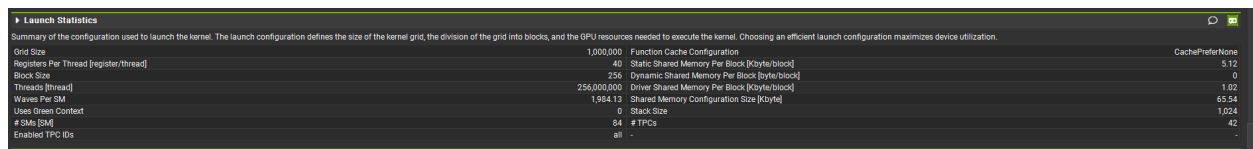
Coarsening factor	Op Time 1	Op Time 2	Total Execution Time
1 (base)	52.1818 ms	31.5358 ms	83.7176 ms
2	50.0386 ms	30.3402 ms	80.3788 ms
4	49.0745 ms	30.0503 ms	79.1248 ms
8	50.9246 ms	30.4776 ms	81.4022 ms

Coarsening factor 4 is the winner. The improvement from 1 to 4 is small (about 5.5% on total) but consistent. Going from 4 to 8 the trend reverses, which is the register pressure showing up. Each thread now has 8 floats in the accumulator array plus all the other live registers, and at some point the compiler runs out of registers for the warp scheduler to keep enough warps in flight, occupancy drops, latency hiding gets worse.

Final Config: Tile Width = 16, Coarse = 4:

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.603567 ms	0.337123 ms	0.94069 ms	0.86
1000	4.86297 ms	3.04312 ms	7.90609 ms	0.886
10000	49.0745 ms	30.0503 ms	79.1248 ms	0.8714

Compared to op_2 baseline ($51.38 + 31.45 = 82.83$ ms), about 4.5% faster on total. Not huge but it's a start and it works with everything else.



Launch Statistics			
Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.			
Grid Size	1,000,000	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	40	Static Shared Memory Per Block [Kbyte/block]	5.12
Block Size	256	Dynamic Shared Memory Per Block [Kbyte/block]	0
Threads [thread]	256,000,000	Driver Shared Memory Per Block [Kbyte/block]	1.02
Waves Per SM	1,984.13	Shared Memory Configuration Size [Kbyte]	65.54
Uses Green Context	0	Stack Size	1024
# SMs [SM]	84	# TPCs	42
Enabled TPC IDs	all		-

The launch stats back up the COARSE=4 sweet spot. Each thread uses 40 registers and the block size is 256, so the per-block register footprint is $40 * 256 = 10,240$ registers. The A40 SM has 65,536 registers, which lets 6 of these blocks fit per SM at once ($10,240 * 6 = 61,440 < 65,536$, but 7 blocks would need 71,680 which doesn't fit). 6 blocks of 256 threads is 1,536 threads per SM, well within the 1,536-thread limit, so the kernel is hitting decent occupancy.

Waves Per SM is 1,984.13, which means there's plenty of work queued up across SMs to keep them all fed. The 1,000,000 grid size at 256 threads each gives 256,000,000 total threads, more than enough to hide latency.

This is what COARSE=4 looks like when register pressure is balanced. Each thread holds COARSE=4 floats in its acc array plus the usual index variables and tile pointers, and the compiler lands at 40 registers, which fits. Going to COARSE=8 would double the accumulator array (8 floats instead of 4), which pushes per-thread registers up, drops blocks per SM, and tanks the warp count available to the scheduler. That's why the COARSE=8 row in the sweep table regressed (50.92 ms vs 49.07 ms on Conv1) even though more coarsening should mean better arithmetic intensity. The register cliff hit before the reuse benefit could pay out.

- **Does this optimization synergize with any other optimizations? How?**

It pairs well with op_1 because the coarsened version has each thread holding COARSE accumulators in registers, and that only stays in registers if the compiler can prove nothing aliases. const restrict is what gives it that.

Works with op_2 because the new inner loop is nested, an outer loop of TILE_WIDTH and an inner loop of COARSE, which means more independent FFMAs per pass. Unrolling can do more.

It works with op_6 in the strongest way. op_6 is essentially coarsening taken further, where each thread accumulates a 2D tile of outputs in registers instead of just COARSE columns. The coarsening sweep here is what convinced me there's headroom for op_6 to dig further into.

- **List your references used while implementing this technique. (you must mention textbook pages at the minimum)**

NVIDIA Corporation. "CUDA C++ Best Practices Guide," Section 9.2.3 Shared Memory and Section 11.1 Arithmetic Instructions.

<https://docs.nvidia.com/cuda/archive/12.8.1/cuda-c-best-practices-guide/index.html>

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 6 (Performance considerations), "Thread coarsening," pp. 138-140.

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 5 (Memory architecture and data locality), "Impact of memory usage on occupancy," pp. 115-117.

Op_4: Using cuBLAS for matrix multiplication

[\[Google Drive link to subfolder op_4\]](#)

- **How does this optimization theoretically optimize your convolution kernel? Expected behavior?**

cuBLAS is NVIDIA's hand-tuned linear algebra library. The idea is that NVIDIA has a whole team writing assembly-level matmul kernels with all the tricks I'd ever care to do (register tiling, double buffering, ldmatrix, tensor cores when possible, perfect occupancy tuning), and they pick the best one for your specific problem size at runtime. So in theory, dropping cuBLAS into the matmul step of an im2col convolution should be at least as fast as anything I write by hand and probably faster.

The catch is that this only works if the matmul is actually big enough to amortize the overhead. cuBLAS picks a kernel using a heuristic that involves a CPU-side dispatch path, sometimes a fallback kernel for non-aligned dimensions, and at small sizes the picker overhead and kernel launch costs can dominate the actual compute. There's also the structural cost that to use cuBLAS at all I have to pre-materialize the unrolled input matrix as a separate buffer, which is a global memory write the fused kernel completely avoids.

I expected this to lose, honestly. Conv2 has Map_out=16 and a batch of 10000 gives Width_unrolled around 240k, so the matmul is 16x49 times 49x2.4M-ish, which is tall and skinny in a way that doesn't suit gemm libraries. Conv1 is even worse with M=4. I figured cuBLAS would be slower on Conv1 and maybe break even or slightly win on Conv2.

- **How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.**

The structure had to change a lot. The fused kernel is gone. Now there are three GPU steps. First an unroll kernel writes the im2col matrix to a separate buffer. Then cublasSgemm does the matmul. Then a permute kernel reorders the output from (Map_out, Batch, Height_out, Width_out) into (Batch, Map_out, Height_out, Width_out).

```
// instead we are doing matrix mul with cuBLAS want row major order

// cublas is col major want C = A * B

cublasHandle_t handle;

cublasCreate(&handle);

const float alpha = 1.0f;

const float beta = 0.0f;

cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, Width_unrolled, Map_out,
Height_unrolled, &alpha, unrolled_matrix, Width_unrolled, device_mask, Height_unrolled, &beta,
matmul_output, Width_unrolled);

cublasDestroy(handle);
```

The dimension swap is because cuBLAS is column-major and our data is row-major, so I'm computing what looks like B times A in cuBLAS's view to get A times B in row-major.

Allocations:

```
cudaMalloc((void*)&unrolled_matrix, (size_t) Height_unrolled * Width_unrolled *
sizeof(float));

cudaMalloc((void*)&matmul_output, (Batch * Map_out * Height_out * Width_out) *
sizeof(float));
```

- **Did the performance match your expectation? Explain why or why not, by analyzing profiling results.**

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	72.1859 ms	1.28734 ms	73.47324 ms	0.86
1000	82.8288 ms	9.30149 ms	92.13029 ms	0.886
10000	99.1184 ms	47.4822 ms	146.6006 ms	0.8714

- **List your references used while implementing this technique. (you must mention textbook pages at the minimum)**

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 16 (Deep learning), "Formulating a convolutional layer as GEMM," pp. 379-384.

NVIDIA Corporation. "cuBLAS Library User Guide," Sections 1, 2.1, and 2.7.

<https://docs.nvidia.com/cuda/archive/12.8.1/cublas/index.html>

Op_5: Fixed point (FP16) arithmetic implementation

[\[Google Drive link to subfolder op_5\]](#)

- **How does this optimization theoretically optimize your convolution kernel? Expected behavior?**

FP16 (half precision) is half the size of FP32. So loads and stores move twice as much data per transaction, shared memory holds twice as many elements per byte, and the dedicated FP16 hardware can do two FMAs in one instruction using the half2 vectorized type. Theoretically that's up to 2x on memory bandwidth and 2x on FMA throughput, plus reduced register pressure from smaller live values.

In practice the wins are usually less than 2x because the kernel is rarely bottlenecked purely on memory or purely on FMA, and there's overhead in converting between FP32 (the input/output types the framework expects) and FP16 (the compute type). On this specific kernel, op_1 NCU showed FMA at around 28% utilization, so the FMA-side win from half2 isn't going to do much. The shared-memory side might help a bit since two tiles of 16x16 halves are 1 KB instead of 2 KB, freeing up shared memory budget and lowering bank-conflict pressure.

I expected this to be approximately neutral, maybe slightly slower because of the conversion overhead at the load and store boundaries. The kernel just isn't bottlenecked on the things FP16 helps with.

- **How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.**

The kernel signature stays the same (still takes float pointers, framework expects FP32). Inside the kernel everything switches to half. The shared tiles are halves, the loads convert FP32 to FP16 on the way in, the inner loop uses half2 to do two FMAs at once, and the result converts back to float at the end.

```
__shared__ __half tileA[TILE_WIDTH][TILE_WIDTH];
__shared__ __half tileB[TILE_WIDTH][TILE_WIDTH];

// Loads convert on the fly
tileA[ty][tx] = __float2half(mask[Row * H_unrolled + a_col]);
```

```
// ...
tileB[ty][tx] = __float2half(in_4d(b, c, h + p, w + q));
```

The inner loop is the interesting part. Instead of one float FFMA per iteration, half2 packs two halves into a 32-bit register and does two FMAs at once with hfma2:

```
__half2 acc2 = __halves2half2(__float2half(0.0f), __float2half(0.0f));
// with half2 can now process 2 K elems at a time
for (int i = 0; i < TILE_WIDTH; i+=2) {
    __half2 a2 = __halves2half2(tileA[ty][i], tileA[ty][i+1]); // 2 adjacent A
elements into half2
    __half2 b2 = __halves2half2(tileB[i][tx], tileB[i+1][tx]); // 2 adj B elements
into half2
    // fp16 multiply
    acc2 = __hfma2(a2, b2, acc2); //fused multiply and add - acc2=a2*b2+acc2
}
// convert back to float
acc += __low2float(acc2) + __high2float(acc2);
```

Note that the float accumulator (acc) is kept in FP32 across tiles. Only the per-tile partial sum stays in FP16. That was a deliberate choice to keep the accumulation precision higher and avoid drift, which is why the final output accuracy only loses one one-thousandth.

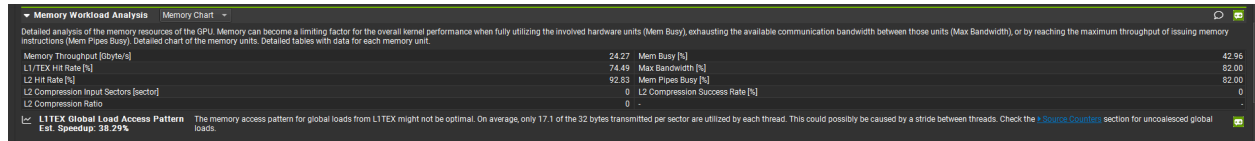
Correctness is essentially identical. Accuracy is 0.86 / 0.886 / 0.8713 vs baseline 0.86 / 0.886 / 0.8714. The 0.0001 drop on batch 10000 is the FP16 precision loss showing up.

- **Did the performance match your expectation? Explain why or why not, by analyzing profiling results.**

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.616074 ms	0.340577 ms	0.956651 ms	0.86
1000	5.30386 ms	3.24894 ms	8.55280 ms	0.886
10000	53.2404 ms	32.2813 ms	85.5217 ms	0.8713

Slightly slower than op_2 baseline ($51.38 / 31.45 = 82.83$ ms). About 3% slower on total.

Direction is what I expected, magnitude is small.



The reason it doesn't help is the same reason op_1 and op_2 didn't help much, the kernel isn't memory bandwidth bound in the way FP16 is built to fix. The op_5 NCU confirms this. L1/TEX hit rate is 74.49% (basically identical to op_1's 74.45%), L2 is 92.83%, memory throughput is 24.27 GB/s, and Mem Busy is 42.96%. The hit rate didn't move because cache behavior is about access patterns, not data size, and the access pattern is the same. The FMA pipe was at 28% utilization in op_1, so doubling FMA throughput with half2 might bring it to maybe 35%, still nowhere near saturation.

The reason it slightly hurts is the conversion overhead. Every load now does an extra float2half on the way in, and the writeback does halves2float and a low/high reduction. Those are cheap individually but they add a few extra instructions to the inner loop and the load path. There's also a side effect visible in the NCU data, the L1TEX uncoalesced load hint went from 25.30% estimated speedup (op_1) to 38.29% (op_5). Each sector is now only delivering 17.1 of 32 bytes useful per thread instead of 22.3. The reason is that halves are half the size of floats, so each 32-byte sector contains twice as many elements, but the stride between elements that threads actually want is unchanged in element count, which means fewer of those packed elements get used per transaction. So FP16 made the per-sector waste slightly worse, and that costs a bit on the load side. With nothing on the win side to offset it, the kernel ends up a few percent slower.

The right place for FP16 in this project is req_1 (Tensor Cores), where the FP16 storage feeds into mma instructions that actually do 16x16x16 matmuls in hardware in one operation. Just converting to FP16 and using half2 on the SIMT path doesn't unlock that, it just changes the precision.

- **Does this optimization synergize with any other optimizations? How?**

It's required for req_1 (Tensor Cores), which is where FP16 actually pays off. Tensor cores eat FP16 fragments and produce FP32 accumulators in hardware, getting the actual order-of-magnitude speedup that half precision is supposed to deliver.

It's mostly neutral with op_1 / op_2 / op_3. The const restrict hint still works on half pointers, the unroll still flattens the inner loop, and coarsening could in principle stack on top.

It conflicts mildly with op_3 in the sense that the half2 inner loop has a strided access pattern (every other K element pairs together) that interacts differently with a coarsened layout. I didn't push on this since the FP16-only version wasn't winning anyway.

- **List your references used while implementing this technique. (you must mention textbook pages at the minimum)**

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 6 (Performance considerations), "Knowing your computation's bottleneck," pp. 145-146.

Harris, M. "Mixed Precision Programming with CUDA 8." NVIDIA Developer Blog.
<https://developer.nvidia.com/blog/mixed-precision-programming-cuda-8/>

NVIDIA Corporation. "CUDA Math API," Sections 4.2, 4.5, 4.6, 15.1, and 15.2.
<https://docs.nvidia.com/cuda/archive/12.8.1/cuda-math-api/index.html>

Op_6: Joint Register and Shared Memory Tiling for matrix multiplication

[\[Google Drive link to subfolder op_6\]](#)

- **How does this optimization theoretically optimize your convolution kernel? Expected behavior?**

This is thread coarsening taken much further. In op_3 each thread computed 4 output columns and kept them in registers. In op_6 each thread computes $U_OUT=16$ outputs (a full column of M dimension) and keeps all 16 in registers. The block is reorganized so it has $T_THREADS=64$ threads in 1D instead of the old 16×16 grid, and the work split is structural.

The key insight is that this turns the matmul into something closer to outer-product accumulation rather than dot-product accumulation. Each outer iteration loads a strip of K elements ($S_K=T_THREADS/U_OUT=4$ elements per thread), and then every thread computes its full U_OUT outputs by hitting that strip. The mask tile M_s gets loaded into shared memory and broadcast to all threads. The N values stay in per-thread registers (n_reg). Then the inner compute is $S_K * U_OUT = 64$ FMAs per thread per outer iteration, all from registers and shared memory, with no further global loads.

Why this is a win compared to op_3. The arithmetic intensity goes way up. In op_3 each loaded element fed into about 16 FMAs per thread per tile. In op_6 each loaded element feeds way more FMAs because the per-thread accumulator array is bigger and the data reuse pattern is deeper. Mask elements are reused $T_THREADS=64$ times. N elements are reused $U_OUT=16$ times. That's much closer to the theoretical limit of how much compute you can extract per byte of memory traffic.

The cost is register pressure. Each thread now holds $U_OUT=16$ floats in the accumulator plus $S_K=4$ floats in n_reg plus index variables, so we're talking roughly 24+ live registers per thread minimum. That can hurt occupancy. But because each block now only has 64 threads instead of 256 (16×16), the per-block register footprint is actually smaller, so occupancy might come out fine or even better.

I expected this to be a big win, somewhere in the 20-30% range over op_3, since the arithmetic intensity argument is strong and the register pressure should be manageable at this size.

- **How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.**

The block layout switches from 2D (16x16) to 1D (64 threads). Each thread is responsible for a full column of U_OUT=16 outputs in the M dimension.

```
#define T_THREADS 64
#define U_OUT 16
#define S_K (T_THREADS / U_OUT) // 64/16= 4
__shared__ float Ms[S_K][U_OUT];
// per thread accumulator
float acc[U_OUT];
for (int u = 0; u < U_OUT; u++) acc[u] = 0.0f;
float n_reg[S_K];
```

The cooperative load of Ms uses tid mapped to (s, u) coordinates, so the 64 threads collectively load all 4*16=64 elements of the Ms strip in one go:

```
int s_load = tid / U_OUT;
int u_load = tid % U_OUT;
int m_row_global = block_row + u_load; // mask row
int m_col_global = k_start + s_load; // mask col
if (m_row_global < Map_out && m_col_global < H_unrolled) {
    Ms[s_load][u_load] = mask[m_row_global * H_unrolled + m_col_global];
} else {
    Ms[s_load][u_load] = 0.0f;
}
```

The N loads each thread does $S_K=4$ of them into its private `n_reg` array. Then the compute kernel is the outer-product style accumulation:

```
for (int s = 0; s < S_K; s++) {
    float n_val = n_reg[s];
    for (int u = 0; u < U_OUT; u++) {
        acc[u] += Ms[s][u] * n_val;
    }
}
```

Grid launch:

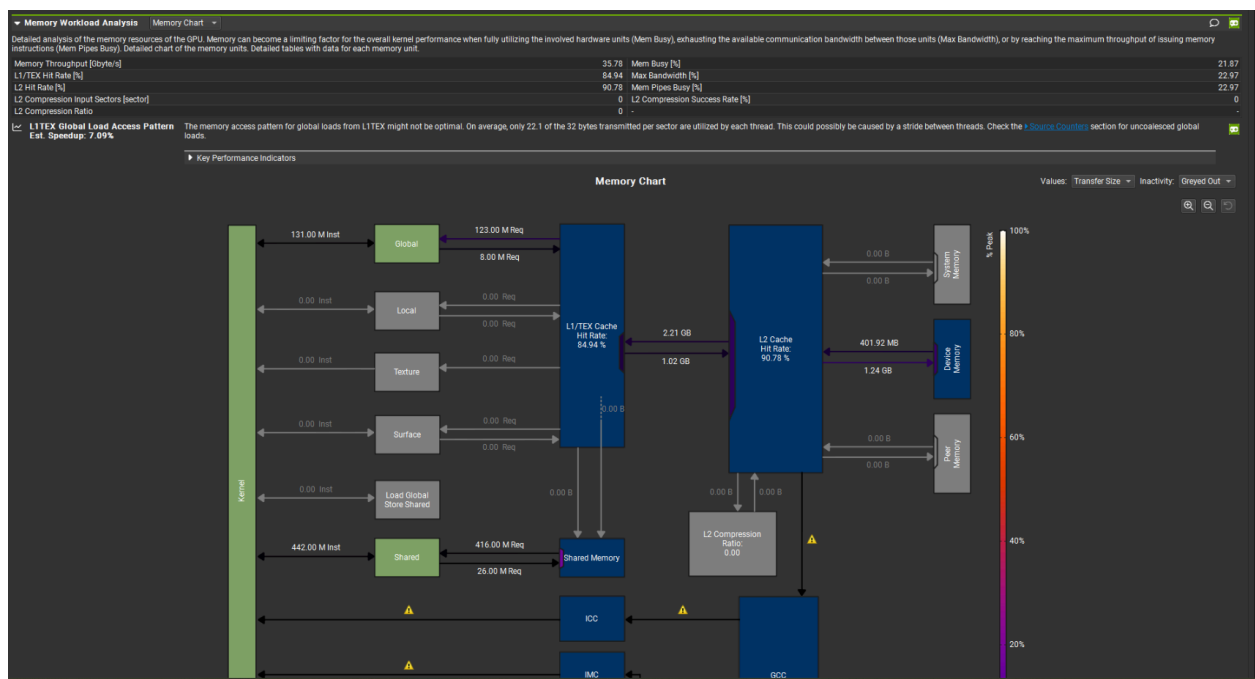
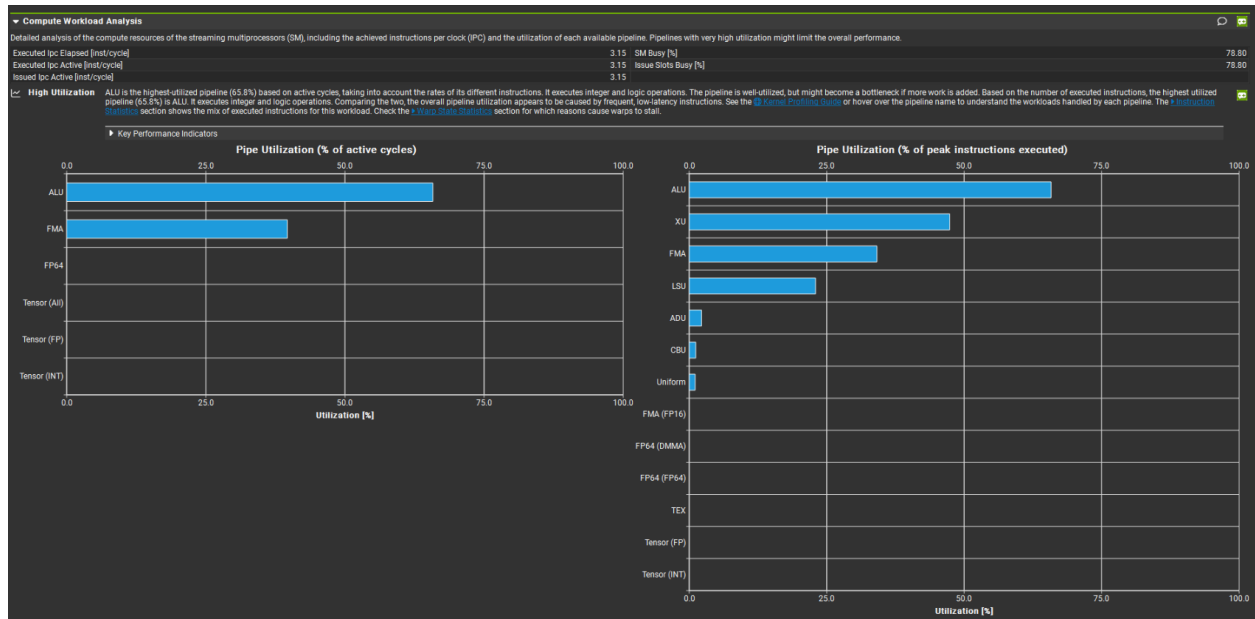
```
dim3 blockDim(T_THREADS, 1, 1);
dim3 gridDim((Width_unrolled + T_THREADS - 1) / T_THREADS,
              (Map_out + U_OUT - 1) / U_OUT, 1);
```

Same Accuracy as baseline as shown below.

- **Did the performance match your expectation? Explain why or why not, by analyzing profiling results.**

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.480157 ms	0.278107 ms	0.758264 ms	0.86
1000	3.56336 ms	2.4635 ms	6.02686 ms	0.886
10000	36.5818 ms	24.2594 ms	60.8412 ms	0.8714

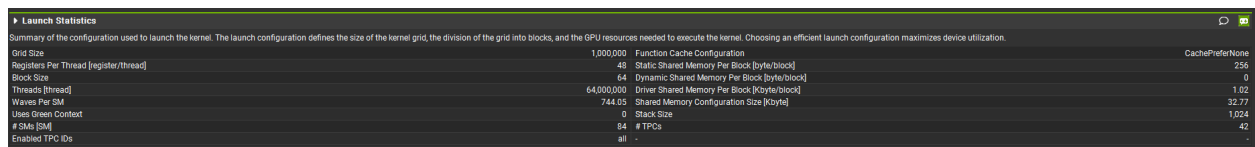
At batch 10000 the total is 60.84 ms vs op_3's 79.12 ms. About 23% faster. Conv1 dropped from 49.07 to 36.58 ms (25% faster) and Conv2 dropped from 30.05 to 24.26 ms (19% faster).



The NCU data confirms what the theory predicted. L1/TEX hit rate jumped from 74.49% in op_5 to 84.94% here. The kernel is keeping more of its working set in cache because each loaded element feeds into way more FMAs before being evicted. Memory throughput went from 24.27 GB/s up to 35.78 GB/s while Mem Busy actually dropped from 42.96% down to 21.87%, which sounds contradictory but makes sense once you think about it. More data is flowing through the cache hierarchy per unit time but the

memory pipes are less congested because the kernel is serving more requests from L1 and L2 instead of fighting at the DRAM boundary. The Memory Chart in the screenshot shows 442M instructions hitting shared memory against 131M hitting the global path, which is the reuse pattern made visible. The uncoalesced load hint that was 38% in op_5 dropped to 7%, the access pattern is way better now. 22.1 of 32 bytes per sector are useful, up from 17.

The compute side moved too. NCU now flags this as "High Utilization" instead of the "Balanced" label op_1 got. ALU is at 65.8% of active cycles (up from 56% in op_3 baseline) and FMA pipe is at about 38% (up from ~28%). SM Busy is 78.8%, IPC is 3.15. The kernel is finally pushing the compute pipes hard enough that further optimization will need to come from elsewhere (template specialization, per-layer dispatch, things in the final stacked submission).



Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	1,000,000	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	48	Static Shared Memory Per Block [byte/block]	256
Block Size	64	Dynamic Shared Memory Per Block [byte/block]	0
Threads [thread]	64,000,000	Driver Shared Memory Per Block [Kbyte/block]	1.02
Waves Per SM	744.05	Shared Memory Configuration Size [Kbyte]	32.77
Ways Green Context	0	Stack Size	1,024
# SMs [SM]	04	# TPCs	42
Enabled TPC IDs	all		-

The Launch Statistics tells the register pressure story. 48 registers per thread, which is high but workable because the block is only 64 threads (1D), so per-block footprint is $64 * 48 = 3072$ registers, well under the 65536-per-SM limit. 744 waves per SM means there are plenty of warps to schedule through latency. The register tiling didn't tank occupancy, which was the main risk going in.

The fact that Conv2 sped up less than Conv1 makes sense too. Conv2 was already getting good reuse out of op_3's COARSE=4 layout, so there was less headroom. Conv1 was the bigger beneficiary because the U_OUT=16 layout still has Map_out=4 worth of waste (12 of 16 output rows zero), but the per-block waste fraction is the same as op_3 while the per-element reuse is way higher.

This is the optimization that actually paid off. The previous ones were compiler hints (op_1, op_2), config tuning (op_3), or library experiments (op_4, op_5). op_6 is a structural change to how the matmul is partitioned, and it's the one that broke through.

- **Does this optimization synergize with any other optimizations? How?**

Yes, with most of them. `const restrict (op_1)` is essentially required, since the per-thread `acc[U_OUT]` register array only stays in registers if the compiler can prove no stores can invalidate it. Without that hint, the compiler would conservatively spill to local memory and the win disappears.

`#pragma unroll (op_2)` helps because the inner double loop ($S_K * U_OUT = 64$ FMAs) is fully known at compile time. Unrolling that exposes 64 independent FMAs per outer iteration that the compiler can schedule freely.

The `op_3` sweep informed parameter choice. `T_THREADS=64` and `U_OUT=16` weren't pulled out of thin air, they're the result of me thinking about what the `op_3` sweep results meant for register pressure and shared memory per block.

- **List your references used while implementing this technique. (you must mention textbook pages at the minimum)**

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 5 (Memory architecture and data locality), "A tiled matrix multiplication kernel," pp. 107-112.

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 5 (Memory architecture and data locality), "Impact of memory usage on occupancy," pp. 115-117.

Kirk, D. B. and Hwu, W. W. Programming Massively Parallel Processors, 4th ed., Chapter 6 (Performance considerations), "Thread coarsening," pp. 138-140.

Lumetta, S. ECE 508 Manycore Parallel Algorithms, Lecture 4: Joint Register and Shared Memory Tiling. <https://lumetta.web.engr.illinois.edu/508/slides/lecture4.pdf>

NVIDIA Corporation. "CUDA C++ Best Practices Guide," Section 11.2 register usage and memory tradeoffs. <https://docs.nvidia.com/cuda/archive/12.8.1/cuda-c-best-practices-guide/index.html>

Final Stacked Submission:

[\[Google Drive link to subfolder final m3-forward.cu\]](#)

The final submission is op_6 plus everything that stacks cleanly on top of it, plus a few extra refinements that were specific to the final config. The starting point is op_6 (joint register and shared memory tiling). On top of that I have const restrict on the read-only pointers from op_1, #pragma unroll on the inner FMA loops and the N load loop from op_2, and a tile size choice informed by the op_3 sweep. The new things added in this version are: templating the kernel on U_OUT and K so the compiler can constant-fold the trip counts and pick a different machine code path per layer (K is always 7, so KK=49 is also constant), per-layer dispatch where Conv1 uses U=4 (since Map_out=4 means anything bigger wastes work) and Conv2 uses U=16, and some address-computation cleanup where b/h/w only depend on Col so they're hoisted out of the per-s loop, and c/p/q in the N load are incremented across s instead of recomputed with div/mod.

The per-layer U dispatch is the biggest of these. In op_6 with U_OUT=16, Conv1 was wasting 12 of every 16 output rows because Map_out=4. In the final version, Conv1 gets its own template instantiation with U=4, which is exactly Map_out, so zero waste. Conv2 keeps U=16 since that fills the M tile cleanly.

Final timing run:

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	2.05433 ms	0.152525 ms	2.206855 ms	0.86
1000	1.29925 ms	1.15645 ms	2.4557 ms	0.886
10000	10.7829 ms	8.405 ms	19.1879 ms	0.8714

Competition Mode:

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
10000	8.93844 ms	8.79947 ms	17.73791 ms	0.8714

Comparing back through the chain. M2 fused baseline at batch 10000 was 82.71 ms total. op_3 was 79.12 ms. op_6 was 60.84 ms. The final version is 19.19 ms. That's about 4.3x faster than the M2 baseline and 3.2x faster than op_6 alone. Conv1 in particular dropped from 36.58 ms (op_6) to 10.78 ms (final), which is the per-layer U dispatch eliminating the zero-padded waste paying off massively. Competition mode shaved off another ~1.5 ms by skipping some of the extra checks.

The Conv1 win is bigger than Conv2's because Conv1 had the most slack to recover, it was the worst case in op_6 thanks to the Map_out=4 padding waste. The templated K=7 constant also lets the compiler fully unroll the kernel-size loops, which wasn't possible when K was a runtime parameter.